Application of the Rjindael Field to Hashing

Kieran Harvie

Copyright ©May 8, 2023. All Rights Reserved.

Contents

1 Introduction **2**
- 1.1 Structure of the Repository . . . 2
- 1.2 Hashing 101 . . . 2
  - 1.2.1 Hashing Terminology . . . 3
- 1.3 Field Theory 101 . . . 3
  - 1.3.1 Finite Fields . . . 4
- 1.4 The Rijndael Field . . . 4

2 Matrix Methods **5**
- 2.1 Gaussian Elimination . . . 5
  - 2.1.1 (pointlessly) Abstract Algebra Preamble . . . 5
  - 2.1.2 (useful) Matrix Preamble . . . 5
  - 2.1.3 The Algorithm . . . 7
- 2.2 Application to Hashing . . . 7

3 Polynomial Methods **8**
- 3.1 Interpolation . . . 8
- 3.2 Evaluation . . . 8
  - 3.2.1 Horner's scheme . . . 8
  - 3.2.2 Estrin's scheme . . . 8
  - 3.2.3 Parallel evaluation . . . 8
- 3.3 Application to Hashing . . . 8

4 Advanced Encryption Standard **9**

A Appendix **10**
- A.1 Generator Table . . . 10
- A.2 Side-Channel . . . 10
- A.3 Matrix Arithmetic . . . 11
  - A.3.1 Diagonalization . . . 11
- A.4 Polynomial Arithmetic . . . 13

Introduction

General introduction This is an accompanying document to the repository.

It is meant to approachable to someone that has freshman understand of math or programing, and hopefully work as an instruction to the other.

And maybe even an ambassador for abstract algebra and cryptography for people that are interested in those things but need practical applications to make them stick.

1.1 Structure of the Repository

1.2 Hashing 101

For our purposes a hash function is an any function that inputs a string and outputs a number. The most direct example is converting the first character to it alphabetical number (A=1, B=2, etc.):

$$\text{hash(Apple)} = 1, \quad \text{hash(Orange)} = 15$$

There are two main applications for hashing: The first is performance, as operations on strings may be more expensive then on numbers so an efficient and well designed hash may allow us to work better on the numbers instead. The second is cryptography, that is the practice of communicating secrets, where we want to hash the string and communicate the number in hopes of keeping the string secrete.

There is a clear tension between these two applications. For performance you'd want as much information about the initial string easily assessable in the hash. But for cryptography you'd want as much information hidden as possible while still allowing communication.

This document aims to serve as an introduction, so wont optimize for either application, but I bring them up this early to wet the readers appetite about why they might care about hashing. **But also to state that the reader should not apply any algorithm here for cryptographic purposes without consulting a professional first.** At least one technique used makes the algorithm susceptible to a side-channel attack, and even if I avoided that technique there would be another one I'm ignorant of.

1.2.1 Hashing Terminology

With applications and warning asides there are some more things the reader should know. First is that in common use the term hash can mean both the hash function and the resulting number from that function.

Secondly there are four general terms worth knowing:

Collision: A collision is when two strings have the same hash. This is broadly undesirable since any operation we want to preform on the hash has to work for both strings.

Hash Table: A common application of hash functions in environments where accessing a look-up table with string keys are inefficient compared to number keys. We basically make a look-up table with number keys and when asked to look-up a string key we hash it first and use the result as the integer key. This will require us to deal with collisions as they come up.

Perfect Hash Function: A perfect hash for a given set of strings is hash function where none of strings collide.

Minimal Perfect Hash Function: A minimal perfect hash for a given set is a perfect hash function where all outputs of the string are less then or equal to the size of the set. In terms of features, but not necessarily performance, this is the ideal hash function as there are no collisions to deal with and no wasted numbers taking up space.

1.3 Field Theory 101

Rational numbers finite fields define the term "field operations" MAKE SURE TO USE THE APPENDIX!!

1.3.1 Finite Fields

history of Galois (wasn't understood in his time not helped by his duel) prime powers unique polynomials

To understand why prime numbers might be imporant consider the following: Assume there is x such that $px = 1 \mod pq$ then we have $x < q$ else we set $x- = q$ till its true. But $x < q$ implies $px < pq$ meaning we never have to take a remainder hence $px = 1$ but $p > 1$ and $x > 1$ means $px > 1$ a contraction.

This type of argument can be use to prove a Bézout's identity

1.4 The Rijndael Field

notation and generators

you can ignore all this theory and just have two operation on bytes that

Matrix Methods

From an abstract algebra a matrix is actually a type of tensor as field[it the underlying object is just a ring it's called a module. Say how walking through a 3×3 examples is useful to show only field operations is used. set spell not working?

MAKE SURE TO USE THE APPENDIX!!

2.1 Gaussian Elimination

2.1.1 (pointlessly) Abstract Algebra Preamble

From an abstract algebra viewpoint, the choice of dual space basis is arbitrary provided they have the right span. We can get such a new dual basis $\mathbf{e}'_{\mathbf{i}}$ from an old basis $\mathbf{e}_{\mathbf{j}}$, provided T is non-singular, through:

$$\mathbf{e}'_{\mathbf{i}} = T(\{\mathbf{e}_{\mathbf{j}}\})$$

So we can transform relationship between the coefficients of tensors in one basis it to another, and if T is affine then the new coefficients are directly transformed through T.

Three particular choices of T are swapping two the basis elements, multiplying a basis element by a non-zero constant, and adding a multiple of basis element index to another.

2.1.2 (useful) Matrix Preamble

Abstract tensor analysis is cool, for the type of people that like that type of thing, but the useful result of the previous subsection is that for matrices a,

b, and M, plus an invertible matrix T we have:

$$M_i^j a^i = b^i \Leftrightarrow T_k^j M_i^k a^i = T_k^j b^k$$

(And this result can be verified directly by multiplying by T or T^{-1} and short-circuiting the previous preamble).

Bellow are examples of the three previous particualr choices of T, hence forth referred to as elementary row operations, for 3×3 matrices:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{31} & m_{32} & m_{33} \\ m_{21} & m_{22} & m_{23} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & a & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ am_{21} & am_{22} & am_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & a & 1 \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} + am_{21} & m_{32} + am_{22} & m_{33} + am_{23} \end{bmatrix}$$

Since these elementary row operations can be represented through matrices their composition can be represented through matrix multiplication. Hence we can build more complex row operations from multiplying a chain of elementary row operations together.

For example say we want a transform T that takes the a 3×3 matrix and makes it first column $[1, 0, 0]$. Assuming $m_{1n} \neq 0$, we have:

$$\begin{bmatrix} m_{11}^{-1} & 0 & 0 \\ 0 & m_{22}^{-1} & 0 \\ 0 & 0 & m_{33}^{-1} \end{bmatrix} \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} 1 & m_{12}m_{11}^{-1} & m_{13}m_{11}^{-1} \\ 1 & m_{22}m_{12}^{-1} & m_{23}m_{12}^{-1} \\ 1 & m_{32}m_{32}^{-1} & m_{33}m_{31}^{-1} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & m_{12}m_{11}^{-1} & m_{13}m_{11}^{-1} \\ 1 & m_{22}m_{12}^{-1} & m_{23}m_{12}^{-1} \\ 1 & m_{32}m_{32}^{-1} & m_{33}m_{31}^{-1} \end{bmatrix} = \begin{bmatrix} 1 & m_{12}m_{11}^{-1} & m_{13}m_{11}^{-1} \\ 0 & m_{22}m_{12}^{-1} - m_{12}m_{11}^{-1} & m_{23}m_{12}^{-1} - m_{13}m_{11}^{-1} \\ 0 & m_{32}m_{32}^{-1} - m_{12}m_{11}^{-1} & m_{23}m_{31}^{-1} - m_{13}m_{11}^{-1} \end{bmatrix}$$

Hence:

$$T = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} m_{11}^{-1} & 0 & 0 \\ 0 & m_{22}^{-1} & 0 \\ 0 & 0 & m_{33}^{-1} \end{bmatrix}$$

Gaussian Elimination is simply using this process to find a matrix T such that $T(M) = I$ by interatvly zeroing the outer, non-diagonal, elments:

$$\begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} \to \begin{bmatrix} 1 & 0 & 0 \\ 0 & m'_{22} & m'_{23} \\ 0 & m'_{32} & m'_{33} \end{bmatrix} \to \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & m''_{33} \end{bmatrix} \to \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

This calculates M^{-1} as the product of matrices representing elemetry row operations. Note that this always works when M is invertable, adafsd but we may need to swap row such that errent zeros put directly into their destination to avoid dividing by zero.

Rrwrite to bring in the opening. Jordan normal form??

2.1.3 The Algorithm

Matrices can easliy be represented as 2d-arrays, elemetry row operations aren't hard to implement either, and the iterative nature of finall equation directly suggests the structure of the algorith. However there is one more observation instead of keeping in memory all the elementry row operation, just directly apply them to the B as we go. augmented matrices are exceleent at facilitating this.

2.2 Application to Hashing

liner and human names $Ma_0 = Ma_1$ for invertable M preprocesing/ offsetting

Polynomial Methods

3.1 Interpolation

Lagrange

3.2 Evaluation

3.2.1 Horner's scheme

$$y_0 = a_n$$
$$y_k = a_{n-k} + xy_{k-1}$$

3.2.2 Estrin's scheme

3.2.3 Parallel evaluation

3.3 Application to Hashing

Minimal perfect hashing.

Advanced Encryption Standard

As alluded to in the introduction one use of the Rijndael field is the Advanced Encryption Standard.

The basis of AES is the Rijndael field and some matrix operations, meaning we inadvertently have all tools necessary to give an overview. Not sure I will implment this in C.

Appendix

A.1 Generator Table

A.2 Side-Channel

I've said in a few places that using generators to simplify field arithmetic makes the implementation susceptible to side-channel attacks. But how? And what is a side-channel attack?

A side-channel is something about the implementation of an algorithm or protocol that lets an attacker get information, rather than something inherently wrong with the algorithm or protocol itself.

The easiest to explain side-channel is the timing. Consider a guarded door and to get access you have to tell the guard a password. You tell the guard two different incorrect passwords but the guard rejects one immediately but waits till you're half way through for the second one. You might reasonably conclude that first password is wholly wrong but that the first half of the second one is correct. By doing this enough times and keeping track of the time the guard rejects the password you can probably figure out the password. You guessing the password has nothing to do with the cryptography of a password but the implementation by the guard, instead they should have waited till you've said the whole password before rejecting it, and hence is a side-channel attack.

Using a look-up table to compute field operations opens you up to a caching side-channel attack.

Because the look-up table is to large to be stored in the CPU registers it

will be accessed from memory. Accessing the table from memory means it's likely to be cashed. The table being cached means someone with the ability to monitor cache access can see what field elements you're using, and maybe work backwards to get some of the string key that was hashed. Hence leaking the string.

This makes using look-up tables unsuitable for application where the string keys are sensitive information.

Before closing this topic I want to bring up that side-channels can include the physical properties of computers as well. Consider a naïve attempt at stopping a timing attack where the guard waits to reject the password but is still noticeably paying less attention. An attacker can still guess the timing from when the guard stops paying attention. Similarly idling a computer till a set amount of time passes for all password validation attempts might cause a change in power consumption or a noticeable reduction in cooling. Hence even power-draw or acoustic properties of a computer are possible side-channels for attacks.

A.3 Matrix Arithmetic

A.3.1 Diagonalization

The description of Gaussian Elimination and the claim that it works with any invertible matrix may make some readers that half remember a linear algebra course nervous. This is because process described is similar to another process, diagonalization, which is not guaranteed to work just because the input matrix is invertible.

A diagonal matrix is one where all non-diagonal elements are zero and transcribed as Diag:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 7 \end{bmatrix} = \mathrm{Diag}(1, 3, 2, 7)$$

Such matrices are important they are the same as the matrix for the elementary row operation of scaling the rows, making matrix multiplication

invoking them conceptually and computationally easy. For example:

$$\mathrm{Diag}(a_0, a_1, \ldots a_n)\,\mathrm{Diag}(b_0, b_1, \ldots b_n) = \mathrm{Diag}(a_0b_0, a_1b_1, \ldots a_nb_n)$$

Diagonalization is the process of taking a matrix M and finding a diagonal matrix D and invertible matrix P such that: [1]

$$MP = PD$$

And this process is important because it enables the simplification of calculations by canceling P's and utilizing diagonal matrix multiplication:

$$\begin{aligned}
M^3 =&(PDP^{-1})^3\\
=&(PDP^{-1})(PDP^{-1})(PDP^{-1})\\
=&PD(P^{-1}P)D(P^{-1}P)DP^{-1}\\
=&PD^3P^{-1}\\
=&P\,\mathrm{Diag}(a_k)^3P^{-1}\\
=&P\,\mathrm{Diag}(a_k^3)P^{-1}
\end{aligned}$$

But unfortunately diagonalization isn't guaranteed for invertible matrices. The conditions of when it works involves something called eigenvalues, which are outside the scope of this document. But what is concerning is how similar this looks to Gaussian Elimination, which given $MA = B$ involves finding an invertible P such that:

$$M'A = PMA = PB$$

Where M' is M after some elementary row operations. What is important to notice is that in diagonalization P is on the right side of M and the left side of D. But during Gaussian Elimination P is on the left side of both M and B. Since matrix multiplication isn't commutative this makes the processes very different and lays nerves to rest.

Readers interesting in learning more about diagonalization and it's uses are encouraged to first loop up eigenvalues and then "The Jordan Normal Form".

[1] Often equivalently written as $M = PDP^{-1}$

A.4 Polynomial Arithmetic